Ising-Edge: budget-exact compression planning for on-device LLMs

Jenish Patel
Snapdragon AI Challenge 2026 · On-Device Generative AI Acceleration & Optimization

30 September 2026

Executive summary. Ising-Edge chooses, for every transformer layer, whether and how strongly to compress it under an exact memory budget. It measures perplexity, fits a quadratic (QUBO) surrogate, and solves it exactly with CP-SAT [Perron and Furnon]. On Qwen3-0.6B, model-chosen plans beat the best of three random plans at equal bytes by 91.6 pp of ΔPPL at $K{=}18$ (95% CI [−94.03, −89.04]), but lose at $K{=}24$. The same QUBO form includes Snapdragon CPU/GPU/NPU placement and latency terms; that part is implemented but has *not* been validated on a device in this repository.

Problem and motivation. Edge devices constrain the memory and bandwidth available to small language models. Low-rank compression saves bytes, but its cost depends on which layers are compressed: compressing one MLP at keep 0.8 costs from +0.45% (layer 8) to +24.67% (layer 27) perplexity (median +2.68%). Costs also compound: a 24-MLP plan measured +275.60% against +57.75% from summing single-layer costs (4.8×); 33 of 40 measured layer pairs are super-additive. A plan must therefore be chosen jointly over layers, in a space of 9,928,631,398 feasible plans for one byte budget.

Approach. (1) *Measure*: an oracle compresses a plan's MLPs (gate, up, down) with activation-aware truncated SVD (keep 0.8 = rank 608, keep 0.9 = rank 704) and scores WikiText-2 [Merity et al., 2017] perplexity (context 512, 40 fixed chunks), keeping per-chunk NLL. (2) *Fit*: a quadratic surrogate of $\log(\mathrm{PPL}/\mathrm{PPL}_{\mathrm{dense}})$ over 48 one-hot plan bits, either a pair-interaction model selected by leave-one-pair-out error (one-shot) or a rank-6 factorization machine [Rendle, 2010] refitted every round (closed loop, in the spirit of FMQA/BOCS [Kitai et al., 2020, Baptista and Poloczek, 2018]). (3) *Solve*: CP-SAT minimises the surrogate with at most one option per layer, an exact byte budget and no-good cuts on measured plans; each round evaluates the optimum plus a Hamming-diverse pick. Chosen plans are re-measured on the full 584-chunk split with paired-bootstrap CIs (4,000 resamples).

Implementation. Python package `isingedge`: `loop/` (space, FM, acquisition, oracle, statistics; evaluated), `twin/` (SnapTwin prefill/decode roofline model fitted by NNLS; unit-tested), `qubo/` (placement QUBO over CPU/GPU/NPU with unit-switch costs; unit-tested), `bench/` and `scripts/` (adb + llama.cpp device runners; not yet run). All results were measured with PyTorch 2.14.1 + CUDA 12.6, fp32, NVIDIA GeForce RTX 3060 Laptop GPU. The test suite collects 56 tests: 52 pass, 3 are skipped without optional torch/onnxruntime, and 1 smoke test fails because two subpackages are missing from this snapshot.

Results.

K	plan ΔPPL (%)	best random (%)	difference (pp)	95% CI (pp)
6	6.76	17.37	−10.61	[−11.24, −9.95]
12	27.05	36.83	−9.78	[−10.59, −8.94]
18	66.90	158.51	−91.61	[−94.03, −89.04]
24	284.18	260.76	+23.42	[+17.24, +29.55]

Matched bytes: K MLPs at keep 0.8, full split. CIs recomputed from per-chunk NLL. Pair, additive and log-additive models selected the same layers.

- **Closed loop** (24 measured plans per run, 3 seeds): the loop matched the one-shot plan in 3/3 seeds at $K{=}12$ (+25.45%) and 2/3 at $K{=}18$ (+64.52%); random search reached only +45.68–47.50% and +115.37–149.85%. At $K{=}18$ seed 2 picked a one-swap neighbour: −1.53 pp on the full split (CI [−2.16, −0.90]) but +0.83 pp on the 40-chunk objective.

- **Surrogate calibration** on 72 held-out random plans: Spearman 0.78–0.81 ($K{=}12$), 0.52–0.63 ($K{=}18$), biased low (−0.25 to −0.18 in log ratio).
- **Solvers**: greedy + swap descent matched the exact optimum in 15/72 rounds; ballistic simulated bifurcation produced infeasible raw samples in 72/72, so exact CP-SAT is used.
- **KL proxy rejected**: Spearman $\rho = 0.64$ (CI 0.35–0.83) against single-layer ΔPPL, below the 0.90 bar.
- **MPO vs SVD**: with layers 2–25 compressed at equal parameters (91.7% of dense), a 3-core MPO [Oseledets, 2011] reached PPL 39.05 vs 53.17 for SVD (dense 21.88); dense reconstruction, accuracy only.

Impact and use case. Given a byte budget, the planner returns per-layer keep fractions whose accuracy has been measured, with confidence intervals, rather than predicted from single-layer tables. The target runtime is llama.cpp [lla] on Snapdragon; Qwen3 (Apache-2.0) and llama.cpp (MIT) keep the stack open source.

Limitations. No Snapdragon latency, memory or energy measurement exists in this repository; SnapTwin and the placement QUBO are unit-tested only, and the plan-export scripts reference data not included here. Results cover one model, one metric (WikiText-2 perplexity) and one runtime (PyTorch fp32), without task accuracy or healing; damage is large (+66.90% at $K{=}18$). The pair model fails for large plans ($K{=}24$), and the loop's gains are small at 24 evaluations. The components are established techniques; the contribution is their integration with an exact byte budget and measured validation.

Future work. Run the scripted llama-bench sweeps on a Snapdragon device (CPU, GPU, NPU); fit SnapTwin on device data and add latency terms; export plans as llama.cpp tensor overrides and measure time-to-first-token, decode rate and memory; add plan-size terms, task accuracy and healing.

References

llama.cpp. `https://github.com/ggml-org/llama.cpp`. MIT License.

Ricardo Baptista and Matthias Poloczek. Bayesian optimization of combinatorial structures. In *International Conference on Machine Learning (ICML)*, 2018.

Koki Kitai, Jiang Guo, Shenghong Ju, Shu Tanaka, Koji Tsuda, Junichiro Shiomi, and Ryo Tamura. Designing metamaterials with quantum annealing and factorization machines. *Physical Review Research*, 2:013319, 2020.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*, 2017.

Ivan V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33(5):2295–2317, 2011.

Laurent Perron and Vincent Furnon. OR-Tools. `https://developers.google.com/optimization`. Google.

Steffen Rendle. Factorization machines. In *IEEE International Conference on Data Mining (ICDM)*, pages 995–1000, 2010.